

Programação em Tempo Real para Sistemas Embarcados

Relatório Projeto Final - Módulo de Interface via Display

Professor: Jes de Jesus Fiais Cerqueira



David Ferrari, João Cerqueira, Vinícius Jesus, Felipe Cunha
Universidade Federal da Bahia, Campus Salvador

Julho, 2026

Sumário

1	Introdução	3
2	Objetivos	4
3	Desenvolvimento	5
3.1	Solução adotada	5
3.2	Geração dos dados	6
3.3	Transformação dos dados	6
3.4	Envio ao display	7
3.5	Pinagem	7
4	Simulações e Validação	8
4.1	Uso da UART2 como recurso de depuração	9
4.2	Teste de troca de telas via UART	10
4.3	Teste do botão virtual PA0 e interrupção EXTI0	10
4.4	Teste de comunicação I2C com o OLED SSD1306	11
4.5	Captura e renderização do framebuffer do OLED	11
4.6	Teste de temporização das tasks	13
4.7	Síntese dos resultados	14
5	Conclusão	15

1 Introdução

Sistemas embarcados aplicados à robótica móvel exigem a integração de diferentes funções, como aquisição de dados, processamento, comunicação, controle e interface com o usuário. Em uma base robótica móvel, como em arquiteturas com várias rodas e diferentes sensores coletando dados, diversas grandezas precisam ser acompanhadas continuamente para que o estado do sistema possa ser devidamente supervisionado. Entre essas grandezas estão as correntes dos motores, as velocidades angulares das rodas, as velocidades lineares da base e as informações de posição.

No contexto da disciplina *Programação em Tempo Real para Sistemas Embarcados*, o projeto final foi organizado como parte de uma solução maior voltada ao controle e monitoramento de uma base robótica móvel. Cada equipe ficou responsável por um módulo específico do sistema, permitindo que o problema fosse dividido em partes menores e mais bem definidas. Neste trabalho, estudar-se-á o desenvolvimento do módulo de interface via display, cuja função é apresentar, de forma organizada, informações relevantes de telemetria do robô em estudo.

O módulo desenvolvido utiliza a placa Black Pill, baseada no microcontrolador STM32F411CEU6, em conjunto com um display OLED SSD1306 monocromático de resolução 128×64 pixels. A comunicação entre o microcontrolador e o display é realizada por meio do barramento I2C, escolha coerente com a proposta do projeto e paralela à aplicações embarcadas presentes no mercado.

A implementação foi construída sobre o FreeRTOS, sistema operacional de tempo real utilizado para estruturar a aplicação em tarefas concorrentes e muito presente no mercado. Essa escolha permite representar, de maneira mais próxima de um sistema real, a existência de diferentes fluxos de dados operando com períodos distintos. No projeto, dados de corrente, velocidade angular e posição são simulados por tabelas de valores e processados por tarefas periódicas. A comunicação entre as tarefas é realizada por filas do FreeRTOS, enquanto o acesso às variáveis globais compartilhadas é protegido por mutexes. Dessa forma, o firmware separa a geração dos dados, a leitura das filas, o processamento das grandezas e a atualização da interface gráfica.

Além da simples exibição de valores no display, o módulo também realiza a transformação das velocidades angulares das três rodas em grandezas associadas ao movimento da base robótica, como as velocidades lineares nos eixos x e y e a velocidade angular do corpo do robô. Essas informações são organizadas em três telas principais: uma tela para velocidade linear e posição, uma tela para velocidades angulares das rodas e velocidade angular do robô, e uma tela para as correntes dos motores. A alternância entre essas telas é feita por meio de um botão conectado ao pino PA0, tratado por interrupção externa, com notificação da tarefa responsável pelo gerenciamento do display.

Como nem todos os sensores e módulos físicos do sistema completo estavam disponíveis para integração, a validação do firmware foi realizada por meio de simulações. O ambiente Renode foi utilizado para executar o firmware compilado para o STM32, observar mensagens de depuração via UART, testar a troca de telas por botão virtual, verificar a comunicação I2C com o dispositivo OLED simulado e capturar o conteúdo real do framebuffer do driver SSD1306. Essa abordagem permitiu validar não apenas a lógica funcional da interface, mas também aspectos importantes de sistemas de tempo real, como periodicidade de tarefas, comunicação entre tarefas, exclusão mútua, interrupções e atualização de periféricos.

A Figura 1 apresenta o módulo de interface dentro da proposta geral do projeto.



Figura 1: Módulo de interface via display desenvolvido no projeto.

2 Objetivos

O objetivo geral deste trabalho é desenvolver e validar um módulo de interface com um display para uma base robótica móvel, utilizando um sistema embarcado baseado no microcontrolador STM32F411CEU6, no sistema operacional de tempo real FreeRTOS e em um display OLED SSD1306 com comunicação I2C.

O módulo tem como finalidade organizar e exibir informações relevantes de telemetria do robô, tais como correntes dos motores, velocidades angulares das rodas, velocidades lineares estimadas e posição. Como os sensores físicos e os demais módulos do sistema completo não estavam integrados ao ambiente de desenvolvimento, os dados de entrada foram simulados por meio de tabelas de valores (lookup table), permitindo validar a lógica de aquisição, processamento, comunicação entre tarefas e atualização da interface gráfica.

De forma mais específica, este trabalho tem como objetivos:

- modelar os dados de corrente, velocidade angular e posição em uma estrutura comum, adequada ao uso em filas do FreeRTOS;
- simular a geração de dados de sensores por meio de tabelas de valores armazenadas em memória;
- implementar tarefas periódicas responsáveis pela geração dos dados simulados em diferentes taxas de atualização;
- utilizar filas do FreeRTOS para desacoplar a produção e o consumo dos dados;
- implementar tarefas leitoras responsáveis por consumir as filas e atualizar o estado mais recente do sistema;
- proteger o acesso às variáveis globais compartilhadas por meio de mutexes, evitando condições de corrida entre as tarefas leitoras e a tarefa de display;
- calcular as velocidades lineares da base robótica a partir das velocidades angulares das três rodas, considerando a cinemática de uma base omnidirecional;
- desenvolver três telas de exibição no display OLED SSD1306: uma para velocidade linear e posição, uma para velocidades angulares e uma para correntes dos motores;
- implementar a troca de telas por meio de botão conectado ao pino PA0, utilizando interrupção externa e notificação de tarefa no FreeRTOS;
- utilizar a UART2 como recurso auxiliar de **depuração** e simulação no ambiente Renode;

- validar o funcionamento do firmware por meio de simulações, verificando a criação das tarefas, a comunicação por filas, a troca de telas, a comunicação I2C com o OLED e a geração do conteúdo gráfico no framebuffer do driver SSD1306.

Assim, o foco do projeto não está no controle completo da base robótica, mas na implementação de um módulo de interface embarcado, capaz de receber ou simular informações do sistema, processar de maneira concorrente e apresentar de forma organizada em um oled.

3 Desenvolvimento

3.1 Solução adotada

Para solucionar o problema, primeiramente definimos como organizaríamos os dados a serem mostrados, afinal em sistemas embarcados os recursos são escassos e uma boa modelagem garante performance, segurança e escalabilidade do projeto, sendo parte fundamental do processo.

Como precisamos mostrar:

- Posição (x, y) ;
- Velocidade angular das rodas (entrada dos sensores);
- Velocidade linear e velocidade angular do robô (grandezas derivadas);
- Corrente em cada motor.

Escolhemos propositalmente construir uma struct de três dimensões, x , y e z , e um campo `TickType_t timestamp` como segue:

```
typedef struct {
    float x;
    float y;
    float z;
    TickType_t timestamp;
} dataset;
```

Dessa forma temos de ganho:

- Uma só definição de fila, todas as filas usam `sizeof(dataset)` como tamanho de elemento, e as tarefas leitoras/produtoras compartilham a mesma lógica;
- Menos código duplicado, as funções de tela, os mutexes e o padrão produtor/consumidor funcionam igual para qualquer grandeza;
- Timestamp comum, toda amostra carrega o tick de geração, útil para rastrear latência/ordem.

Como o robô tem 3 rodas, temos 3 valores de corrente e 3 valores de velocidade angular, que ocupam integralmente os campos x , y e z . Já a posição usa apenas x e y , ficando o campo z sem uso nesse caso. O reaproveitamento acontece no dataset de velocidade linear, grandeza derivada da transformação: os campos x e y guardam V_x e V_y , e o campo z guarda a velocidade angular do corpo do robô (ω), calculada na mesma

etapa. Dessa forma nada é perdido e conseguimos representar todo o estado do robô para então ser enviado ao display.

Definida a modelagem dos dados, a arquitetura do sistema segue o padrão produtor/consumidor sobre as filas do FreeRTOS. Para cada grandeza existe uma tarefa produtora, que gera as amostras na taxa do respectivo sensor e as deposita em uma fila dedicada, e uma tarefa consumidora, que drena essa fila, aplica as transformações necessárias e publica o valor mais recente em uma variável global protegida por mutex, de onde a tarefa de display faz a leitura. Esse desacoplamento permite que cada lado opere em seu próprio ritmo, os produtores rodam com prioridades altas e períodos curtos, enquanto os consumidores e a interface rodam mais lentamente, sem que um bloqueie o outro. As filas absorvem rajadas de produção e, por serem estruturas thread-safe do próprio RTOS, dispensam sincronização manual no hand-off entre tarefas, a exclusão mútua explícita fica restrita ao ponto em que consumidores e display compartilham o estado final do robô.

3.2 Geração dos dados

Como o projeto não dispõe dos sensores físicos, os dados de entrada são simulados por meio de tabelas de consulta (lookup tables). Um script em Python (`generate-lookup-table.py`) gera, em tempo de compilação, nove vetores de 100 posições com valores aleatórios dentro de faixas realistas para cada grandeza: corrente de 0 a 4 A, velocidade angular de 0 a 10π rad/s e posição de 0 a 1000 cm. O resultado é escrito diretamente no arquivo `lookuptable.c`, que é compilado junto ao firmware, evitando qualquer custo de geração em tempo de execução.

Dentro do sistema embarcado, três tarefas produtoras percorrem esses vetores de forma circular, simulando um fluxo contínuo de amostras. Cada tarefa monta um dataset com os três valores da sua grandeza e o tick atual do RTOS como timestamp, e o envia para a fila correspondente com `xQueueSendToBack` sem bloqueio. Os períodos foram definidos para refletir a dinâmica de cada sensor: corrente a cada 1 ms (fila de 50 elementos), velocidade angular a cada 10 ms (fila de 30) e posição a cada 100 ms (fila de 5). A periodicidade é garantida por `vTaskDelayUntil`, que mantém o período fixo sem acúmulo de atraso. Caso alguma fila encha, o dado é descartado e o LED da placa sinaliza a condição de overflow.

3.3 Transformação dos dados

A transformação principal ocorre na tarefa leitora de velocidade angular. As lookup tables fornecem as velocidades angulares das três rodas (M_1 , M_2 , M_3), mas a interface deve exibir a velocidade do robô como corpo. Para isso aplica-se a cinemática direta de uma base omnidirecional de três rodas dispostas a 120° (ângulos α de 0° , 120° e 240°), com raio de roda $r = 5$ cm e distância do centro à roda $L = 10$ cm:

$$\begin{aligned} V_x &= r \cdot \frac{2}{3} \cdot [-\sin(\alpha_1)M_1 - \sin(\alpha_2)M_2 - \sin(\alpha_3)M_3] \\ V_y &= r \cdot \frac{2}{3} \cdot [\cos(\alpha_1)M_1 + \cos(\alpha_2)M_2 + \cos(\alpha_3)M_3] \\ \omega &= \frac{r(M_1 + M_2 + M_3)}{3L} \end{aligned}$$

Os senos e os cossenos são pré-calculados como constantes na inicialização da tarefa, evitando chamadas trigonométricas repetidas a cada amostra. O resultado é armazenado em um `dataset` de velocidade linear, com V_x e V_y nos campos x e y , e a velocidade angular do corpo ω reaproveitando o campo z . Cada tarefa leitora esvazia sua fila em laço e grava o valor mais recente na variável global de snapshot correspondente, sempre protegida por mutex (padrão take, escreve, give, com timeout de 1 ms), garantindo exclusão mútua com a tarefa de exibição.

3.4 Envio ao display

A comunicação com o display OLED SSD1306 é feita pelo barramento I2C1 a 400 kHz (pinos PB6/SCL e PB7/SDA), com o display no endereço 0x3C. O driver mantém um framebuffer de 1 KB em RAM (128×64 pixels, 1 bit por pixel); as funções de desenho de texto e primitivas escrevem apenas nesse buffer, e a função `SSD1306_UpdateScreen` transfere o buffer completo ao display via HAL. Isso reduz o número de transações I2C e evita cintilação.

A tarefa `vDisplayManager`, de menor prioridade do sistema, gerencia três telas: velocidade linear e posição, velocidade angular, e correntes dos motores. Ela bloqueia em `ulTaskNotifyTake` com timeout de 699 ms. Se o timeout expira, apenas os valores da tela atual são atualizados, lendo as variáveis globais sob mutex. Se uma notificação chega, a tela avança ciclicamente e é redesenhada por completo.

A notificação vem do botão KEY (PA0), tratado por interrupção externa. O pressionamento gera uma borda de descida que dispara a EXTI0; o handler `EXTI0_IRQHandler` delega ao HAL, que invoca o callback `HAL_GPIO_EXTI_Callback` reimplementado na aplicação. Nele aplica-se um debounce por software de 200 ms, necessário porque o botão da placa não possui filtro de hardware, e em seguida a tarefa de display é acordada com `vTaskNotifyGiveFromISR`, seguida de `portYIELD_FROM_ISR` para troca de contexto imediata. Esse é o padrão de interrupção diferida: a ISR faz o mínimo e o trabalho pesado fica em contexto de tarefa. A EXTI0 foi configurada com prioridade 5, o valor mais urgente permitido pelo `configMAX_SYSCALL_INTERRUPT_PRIORITY` do FreeRTOS para ISRs que chamam APIs `FromISR`.

Além da EXTI0, o sistema usa as interrupções de infraestrutura do RTOS: o SysTick gera o tick do escalonador, o par `PendSV/SVC` realiza as trocas de contexto e o TIM1 serve como base de tempo do HAL, já que o SysTick fica dedicado ao FreeRTOS. Quanto aos handlers de objetos do RTOS, o sistema mantém três `QueueHandle_t` (uma fila por grandeza), quatro `SemaphoreHandle_t` (um mutex por variável global compartilhada) e um `TaskHandle_t` da tarefa de display, usado como alvo da notificação enviada pela interrupção do botão.

3.5 Pinagem

Utilizando os equipamentos citados anteriormente, a pinagem do projeto ficou como segue na imagem.

- **LED (PC13):** Alimenta o LED indicador da placa. No firmware, é apagado quando ocorre overflow em alguma das filas FreeRTOS.
- **BTN (PA0):** Botão KEY da Black Pill. Dispara a interrupção EXTI0 (borda de descida, pull-up interno) para trocar a tela exibida no OLED.

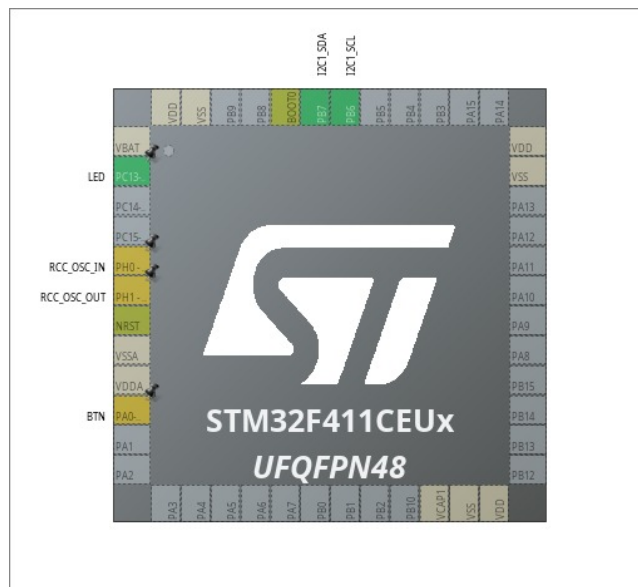


Figura 2: Planejamento dos pinos para o STM32F411CEUx

- **I2C1_SCL (PB6):** Pino de clock da interface I2C1, essencial para a comunicação com o display OLED SSD1306.
- **I2C1_SDA (PB7):** Pino de dados da interface I2C1, essencial para a comunicação com o display OLED SSD1306.
- **RCC_OSC_IN / RCC_OSC_OUT (PH0 / PH1):** Pinos do oscilador externo (cristal).

4 Simulações e Validação

A validação do módulo de interface foi realizada com o *Renode*, utilizado para executar o firmware compilado para o microcontrolador STM32F411CEU6 em um ambiente simulado. Essa etapa foi importante porque permitiu verificar o comportamento do sistema mesmo sem a integração completa com todos os sensores e periféricos físicos da base robótica.

As simulações tiveram como objetivo validar o fluxo principal do firmware, desde a geração dos dados simulados até a atualização da interface gráfica. De forma geral, o caminho verificado foi:

```
lookuptable.c
-> tasks geradoras FreeRTOS
-> filas
-> tasks leitoras
-> mutexes
-> variáveis globais
-> gerenciador de display
-> driver SSD1306
```



```
-> I2C1
-> OLED simulado no endereço 0x3C
```

Além disso, a UART2 foi utilizada como recurso auxiliar de depuração. É importante destacar que a UART2 não substitui o display OLED no projeto. Ela foi usada apenas para observar, no terminal, o comportamento interno do firmware durante a simulação, permitindo acompanhar a criação das tarefas, os valores exibidos nas telas, os comandos de troca de tela e os resultados dos testes. Isso foi extremamente importante, principalmente pela ausência de simuladores específicos para esse trabalho, como a falta do OLED.

4.1 Uso da UART2 como recurso de depuração

A primeira etapa de validação consistiu em utilizar a UART2 para observar a execução do firmware no Renode. A UART foi conectada a uma porta TCP, permitindo a visualização das mensagens de debug por meio de um terminal no computador. O firmware foi iniciado no Renode com o script de simulação e, em outro terminal, a saída serial foi acompanhada com o comando:

```
nc localhost 12345
```

Na inicialização, a UART permitiu confirmar a criação das principais tarefas do FreeRTOS. Foram observadas mensagens indicando a criação da task gerenciadora do display, das tasks produtoras de dados, das tasks leitoras de filas e da task auxiliar de comandos de simulação:

```
INIT RTOS | heap before tasks=30784
CREATE displayManager=1
CREATE gerarQueueCorrente=1
CREATE gerarQueueVelAngular=1
CREATE gerarQueuePosicao=1
CREATE queueCorrenteReader=1
CREATE queueVelAngularReader=1
CREATE queuePosicaoReader=1
CREATE simCommandReader=1
```

Esse resultado confirmou que o FreeRTOS foi iniciado corretamente e que o heap disponível foi suficiente para criar os objetos principais da aplicação. Também foi possível observar, pela UART, os valores que estavam sendo enviados para cada uma das telas do display.

Na SCREEN1, por exemplo, foram exibidas as velocidades lineares calculadas e os valores de posição:

```
SCREEN1 | Vx=24.0 | Vy=-142.2 | Px=472.67 | Py=737.60
SCREEN1 | Vx=-215.9 | Vy=145.5 | Px=942.74 | Py=125.76
```

Essas mensagens permitiram verificar que os dados estavam sendo gerados, processados pelas tasks leitoras, armazenados nas variáveis globais protegidas por mutexes e finalmente acessados pela task de display.

SCREEN1	Vx=-141.2	Vy=33.9	Px=708.22	Py=5.65
SCREEN1	Vx=68.7	Vy=-160.3	Px=85.35	Py=314.57
SCREEN2	M1=15.2	M2=20.0	M3=3.7	W=9.74
SCREEN2	M1=28.3	M2=31.1	M3=8.6	W=17.00

Figura 3: Mudança nos screens

4.2 Teste de troca de telas via UART

Após confirmar a execução básica do firmware, foi realizado um teste de troca de telas por meio da UART2. Para isso, a task auxiliar `vTaskSimCommandReader` foi utilizada para receber comandos de simulação. Ao digitar o caractere `n`, o firmware enviava uma notificação à task `vDisplayManager`, solicitando a troca para a próxima tela.

A sequência esperada era:

```
SCREEN1 -> SCREEN2 -> SCREEN3 -> SCREEN1
```

Durante o teste, a aplicação iniciou na `SCREEN1`. Após o envio do comando `n`, a tela mudou para a `SCREEN2`, responsável por apresentar as velocidades angulares das três rodas e a velocidade angular do corpo do robô. Um novo envio do comando levou à `SCREEN3`, que apresenta as correntes dos motores. Ao enviar o comando novamente, o sistema retornou para a `SCREEN1`.

Esse teste confirmou que a lógica de alternância entre telas funcionava corretamente e que a task gerenciadora do display respondia às notificações recebidas. No entanto, esse mecanismo foi utilizado apenas como ferramenta de simulação e depuração. No funcionamento previsto para o módulo, a troca de telas é realizada pelo botão conectado ao pino PA0.

4.3 Teste do botão virtual PA0 e interrupção EXTI0

Depois da validação da troca de telas via UART, foi realizado um teste mais próximo do comportamento esperado no hardware físico. O botão foi configurado no pino PA0, utilizando interrupção externa pela linha EXTI0. No Renode, o botão virtual foi acionado por meio dos comandos:

```
sysbus.gpioPortA.UserButton Press
sysbus.gpioPortA.UserButton Release
```

O fluxo esperado para esse teste era:

```
botão virtual
-> GPIOA/PA0
-> EXTI0
-> EXTI0_IRQHandler()
-> HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0)
-> HAL_GPIO_EXTI_Callback(GPIO_PIN_0)
-> vTaskNotifyGiveFromISR()
-> vDisplayManager
-> troca de tela
```

A interrupção foi tratada no arquivo `stm32f4xx_it.c`, por meio da função:

```
void EXTI0_IRQHandler(void)
{
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0);
}
```

No callback `HAL_GPIO_EXTI_Callback`, foi implementado um debounce por software de 200 ms, evitando que ruídos ou múltiplas transições rápidas do botão gerassem várias trocas de tela indevidas. A interrupção não executa diretamente a atualização da tela; ela apenas notifica a task de display por meio da função `vTaskNotifyGiveFromISR`. Essa abordagem é adequada para sistemas de tempo real, pois mantém a rotina de interrupção curta e transfere o processamento da interface para o contexto da task.

O teste confirmou que o botão virtual do Renode acionou corretamente a interrupção `EXTI0` e que a task `vDisplayManager` realizou a troca de tela. Assim, foi validado o mecanismo real previsto para a interação do usuário com o módulo.

4.4 Teste de comunicação I2C com o OLED SSD1306

No ambiente Renode, o OLED foi representado por um dispositivo I2C simulado no endereço `0x3C`, o mesmo endereço utilizado pelo driver `SSD1306` no firmware. Para verificar se as chamadas de atualização do display estavam de fato chegando ao barramento I2C, foram adicionados contadores no driver:

```
ssd1306_i2c_ok_count
ssd1306_i2c_err_count
```

Esses contadores registram, respectivamente, transmissões I2C bem-sucedidas e transmissões com erro. Durante o teste, os contadores foram lidos diretamente da memória simulada do microcontrolador. O resultado observado foi:

```
ssd1306_i2c_ok_count   = 0x00010701
ssd1306_i2c_err_count  = 0x00000000
```

Convertendo o primeiro valor para decimal:

$$0x00010701 = 67329$$

Portanto, durante a simulação, foram registradas 67329 escritas I2C bem-sucedidas e nenhum erro de comunicação. Esse resultado indicou que o firmware estava executando o caminho real de atualização do display, passando pelo driver `SSD1306`, pela `HAL I2C`, pelo barramento `I2C1` e pelo dispositivo OLED simulado no Renode.

4.5 Captura e renderização do framebuffer do OLED

Uma limitação da configuração utilizada no Renode é que o OLED `SSD1306` não é renderizado automaticamente como uma tela gráfica de 128×64 pixels. O periférico simulado permite validar a comunicação no barramento I2C, mas não mostra visualmente os pixels enviados para o display.

Para contornar essa limitação, foi adotada uma estratégia complementar: capturar diretamente o framebuffer do driver `SSD1306` na memória simulada. O display possui resolução de 128×64 pixels e é monocromático. Como cada pixel é representado por 1 bit, o tamanho total do framebuffer é:

$$128 \times 64 \div 8 = 1024 \text{ bytes}$$

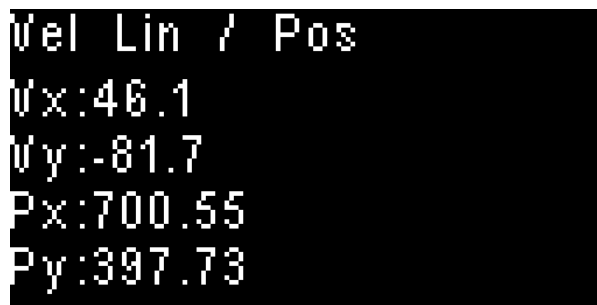
No driver, o framebuffer é declarado como:

```
static uint8_t ssd1306_buffer[SSD1306_BUFFER_SIZE];
```

Após localizar o endereço desse buffer no arquivo ELF gerado pela compilação, foi criado um script para ler os 1024 bytes correspondentes diretamente da memória simulada. Em seguida, outro script em Python interpretou esse conteúdo no formato utilizado pelo SSD1306, em que cada byte representa 8 pixels verticais, e gerou imagens PNG das telas.

Essa etapa foi importante porque as imagens geradas não vieram da UART, mas sim do conteúdo real do framebuffer preparado pelo driver SSD1306. Portanto, a validação visual confirmou que o firmware estava desenhando corretamente as telas que seriam enviadas ao OLED físico.

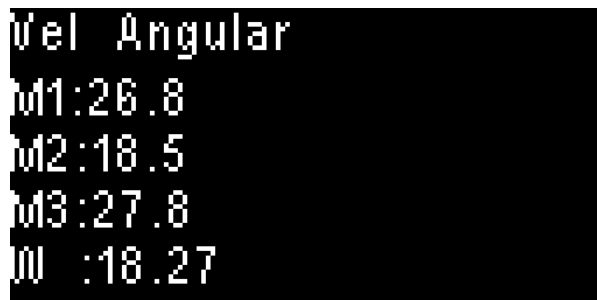
A Figura 4 apresenta uma captura da SCREEN1, contendo velocidade linear e posição.



```
Vel Lin / Pos
Vx:46.1
Vy:-81.7
Px:700.55
Py:397.73
```

Figura 4: Captura renderizada do framebuffer da SCREEN1, exibindo velocidade linear e posição.

A Figura 6 apresenta uma captura da SCREEN2, contendo as velocidades angulares das rodas e a velocidade angular do robô.



```
Vel Angular
M1:26.8
M2:18.5
M3:27.8
W :18.27
```

Figura 5: Captura renderizada do framebuffer da SCREEN2, exibindo velocidades angulares.

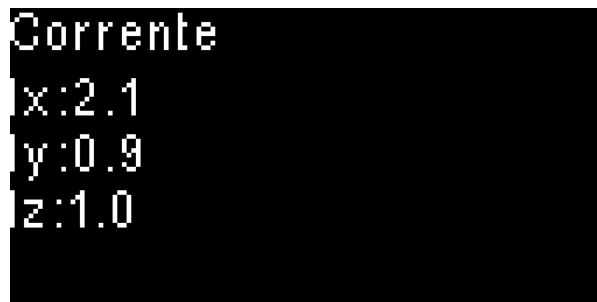


Figura 6: Captura renderizada do framebuffer da SCREEN3, exibindo correntes.

4.6 Teste de temporização das tasks

Além dos testes funcionais, foi realizada uma simulação específica para verificar a temporização das tasks do FreeRTOS. Para isso, foram adicionados contadores globais ao firmware, incrementados a cada execução das tasks geradoras, leitoras e da task de display. Também foi criada uma task auxiliar, chamada `vTaskTimingMonitor`, responsável por imprimir, a cada 10 segundos, a quantidade de execuções observada em cada task.

A janela de observação utilizada foi definida por:

```
#define TIMING_MONITOR_WINDOW_MS 10000
```

Os períodos configurados no firmware são apresentados na Tabela 1.

Task	Período	Execuções esperadas em 10 s
Geração de corrente	1 ms	10000
Geração de velocidade angular	10 ms	1000
Geração de posição	100 ms	100
Leitura da fila de corrente	25 ms	400
Leitura da fila de velocidade angular	100 ms	100
Leitura da fila de posição	125 ms	80
Atualização do display	700 ms	aproximadamente 14 ativações nominais

Tabela 1: Períodos configurados para as principais tasks do firmware.

Durante a simulação, foram observadas duas janelas consecutivas de 10 segundos. Os resultados obtidos foram:

```
TASK TIMING | janela=10000 ms
GEN | corrente=10006 | velAngular=1001 | posicao=100
READ | corrente=400 | velAngular=100 | posicao=81
DISP | display=5

TASK TIMING | janela=10000 ms
```

```

GEN   | corrente=10005 | velAngular=1001 | posicao=100
READ  | corrente=401   | velAngular=100  | posicao=80
DISP  | display=4

```

A Tabela 2 compara os valores esperados com os valores observados.

Task	Esperado em 10 s	Observado
Geração de corrente	10000	10005–10006
Geração de velocidade angular	1000	1001
Geração de posição	100	100
Leitura da fila de corrente	400	400–401
Leitura da fila de velocidade angular	100	100
Leitura da fila de posição	80	80–81
Atualização completa do display	aproximadamente 14 ativações nominais	4–5 atualizações completas

Tabela 2: Resultado observado no teste de temporização das tasks.

Os resultados mostram que as tasks geradoras e leitoras apresentaram contagens compatíveis com os períodos definidos no firmware. A geração de corrente, configurada para ocorrer a cada 1 ms, executou cerca de 10000 vezes em 10 segundos. A geração de velocidade angular, com período de 10 ms, executou cerca de 1000 vezes. A geração de posição, com período de 100 ms, executou exatamente 100 vezes nas janelas observadas.

Da mesma forma, as tasks leitoras apresentaram comportamento compatível com seus períodos. A task leitora de corrente, configurada para 25 ms, executou cerca de 400 vezes em 10 segundos. A task leitora de velocidade angular executou 100 vezes, e a task leitora de posição executou entre 80 e 81 vezes.

A task de display apresentou uma taxa efetiva menor do que a estimativa nominal baseada apenas no valor de `REFRESH_SCREEN`. Isso ocorre porque o contador foi incrementado após a execução completa da função `dataScreen()`, que inclui a escrita dos textos no framebuffer, a chamada de `SSD1306_UpdateScreen()`, as transmissões I2C e as mensagens de debug pela UART. Portanto, o valor medido representa o refresh completo da interface, e não apenas o tempo de espera configurado no `ulTaskNotifyTake`.

Esse comportamento é aceitável para o módulo desenvolvido, pois a interface OLED apresenta informações textuais de telemetria e não exige alta taxa de atualização visual. O resultado também evidencia uma característica típica de sistemas embarcados: tasks de aquisição e processamento podem operar com períodos menores, enquanto a interface com o usuário pode ser atualizada em uma frequência mais baixa.

4.7 Síntese dos resultados

A Tabela 3 resume os principais testes realizados durante a validação do módulo.

De forma geral, as simulações permitiram validar o funcionamento do módulo de interface em diferentes níveis. A UART2 possibilitou a observação do comportamento interno do firmware. O botão virtual demonstrou que a troca de telas pode ser acionada por interrupção externa, como previsto para o hardware físico. Os contadores I2C

Teste	Objetivo	Resultado
UART2 de depuração	Observar a criação das tasks e os valores enviados às telas.	Aprovado
Troca de telas via UART	Validar a lógica do gerenciador de display.	Aprovado
Botão PA0/EXTIO	Validar interrupção externa, callback e notificação de task.	Aprovado
Comunicação I2C/OLED	Confirmar transmissões ao display SSD1306 no endereço 0x3C.	Aprovado
Framebuffer SSD1306	Capturar e renderizar o conteúdo real preparado para o OLED.	Aprovado
Temporização das tasks	Verificar se as tasks periódicas executam em taxas compatíveis com os períodos definidos.	Aprovado

Tabela 3: Resumo das simulações realizadas no projeto.

confirmaram que o driver SSD1306 realizou transmissões bem-sucedidas para o OLED simulado. A captura do framebuffer permitiu validar visualmente as telas geradas pelo firmware. Por fim, o teste de temporização demonstrou que as tasks geradoras e leitoras mantiveram comportamento compatível com os períodos definidos no projeto.

Assim, mesmo sem a integração completa com sensores reais, a validação em ambiente simulado forneceu evidências consistentes de que o módulo de interface executa corretamente suas funções principais: receber ou simular dados, processá-los por meio de tasks e filas do FreeRTOS, proteger o acesso ao estado global com mutexes, alternar entre telas e atualizar a interface OLED.

5 Conclusão

O projeto atingiu o objetivo de coletar, transformar e exibir dados de telemetria de uma base robótica omnidirecional em um sistema de tempo real. A arquitetura produtor/consumidor com filas do FreeRTOS mostrou-se adequada para desacoplar taxas de amostragem distintas (de 1 ms a 100 ms) do ritmo lento da interface, enquanto os mutexes garantiram acesso consistente ao estado compartilhado sem condições de corrida.

Entre os principais aprendizados destacam-se o dimensionamento conjunto de filas e períodos de tarefas, cuja inconsistência inicial causava perda de dados por overflow; a importância de inicializar corretamente todos os objetos de sincronização, já que um mutex não criado leva a comportamento indefinido; e o tratamento correto de interrupções em conjunto com o RTOS, respeitando os limites de prioridade para uso das APIs `FromISR` e aplicando debounce por software. A substituição da troca de telas por tempo pela troca por botão via interrupção externa também evidenciou o padrão de interrupção diferida, que mantém as ISRs curtas e previsíveis, característica essencial em sistemas de tempo real.

Como trabalhos futuros, o módulo pode ser integrado aos módulos das demais equipes, substituindo as lookup tables por dados reais de sensores, e a interface pode evoluir para exibir alertas e históricos das grandezas monitoradas.